QECTOR Workbench - Complete API Reference

Workbench 0.5.2 - Backend qector_decoder_v3 0.7.0 (min 0.7.0) - 56 MCP tools - 16 decoders - 10 code families

Decoder package: qector_decoder_v3 0.7.0 (bundled wheel, activated offline on first launch)

Generated 2026-08-02T01:56:12+00:00Z

This manual is generated from the live application source so every tool name, decoder kind, code family, and function signature matches the running build exactly.

Code families

Family	Parameter	Type	Graphlike	Decoders	Notes
repetition	distance	int	yes	all (16)	1D chain parity-check code.
ring	distance	int	yes	all (16)	Periodic 1D chain.
rotated_surface	distance	int	yes	all (16)	Standard rotated surface code.
unrotated_surface	distance	int	yes	15 (lookup_table refused >20 checks)	Square lattice surface cod
toric	distance	int	yes	15 (lookup_table refused >20 checks)	Toric code with periodic boundaries.
heavy_hex	distance	int	yes	all (16)	IBM heavy-hex lattice.
hypergraph_product	distance	int	yes	all (16)	CSS from repetition seed; graphlike.
bicycle	circulant size	int	no	all (16)	qLDPC bicycle code; graphlike enough for all decoders.
bivariate_bicycle	preset index	int	no	13 (excludes union_find, fast_union_find, lookup_table)	IBM
color_code	triangular size	int	no	colour_code, bp_osd, blossom, hybrid, auto_router	Triangular & 2

Decoder kinds

Kind	Description	Options	Compatibility
union_find	Fast approximate matching via union-find.	bp_method, osd_order ignored	graphlike only
fast_union_find	Faster union-find variant; approximate, higher LER.	-	graphlike only
blossom	Weight-optimal exact MWPM; matches PyMatching LER.	-	all
sparse_blossom	Region-growing near-optimal matching; not exact.	-	graphlike only
sparse_blossom_radix_neighbors	RadixHeap k-NN candidate edge discovery variant of region-growing matching.		
bp_osd	Belief propagation + ordered statistics for LDPC/qLDPC.	bp_method, osd_order, error_rate	all
auto	Self-selecting AutoDecoder.	-	graphlike only
hybrid	Combines multiple strategies; chooses per problem.	-	graphlike only
lookup_table	Exhaustive syndrome-to-correction table; refused above 20 checks.	-	small codes only
predecoded	Fast pre-decoding pass before matching.	-	graphlike only
auto_router	Policy decoder: matching for graphlike, bp_osd for qLDPC. Universally compatible.	-	all
hybrid_cascade	Union-Find pre-filter + Blossom/BP-OSD escalation; exposes cascade stats.	escalation, err	
gnn_belief_matching	GNN-guided weighted matching with faithfulness fallback.	gnn_hidden_size, gnn_n_laye	
belief_matching	BP posteriors reweight exact Blossom matching; faithfulness fallback.	bp_method, osd_ord	
two_stage	Two-stage decode pipeline (fast stage + exact escalation).	escalation	graphlike only
ambiguity_cluster	Cluster decoding for high-noise/non-graphlike degenerate syndromes.	-	non-graphlike
colour_code	Native colour-code decoder over undecomposed detector error models.	-	color_code family on

Decoder options

Key	Type	Applies to	Description
`bp_method`	string	`bp_osd`, `belief_matching`	`"exact"` or `"min_sum"`.
`osd_order`	int	`bp_osd`, `belief_matching`	`0`, `1`, or `2`. Higher is slower/more accurate.
`error_rate`	float	all	Physical error probability used to weight edges or set BP priors.
`escalation`	string	`hybrid_cascade`	`"blossom"` or `"bp_osd"`.
`max_accept_weight`	int	`hybrid_cascade`	Maximum syndrome weight accepted by the pre-filter.
`gnn_hidden_size`	int	`gnn_belief_matching`	Hidden dimension of the GNN.
`gnn_n_layers`	int	`gnn_belief_matching`	Number of GNN message-passing layers.

Unknown keys are ignored with a warning; missing keys use backend defaults.

Measured data

All figures below were measured on this machine (seeded, n=50, p=0.05, rotated_surface d=5). They are workload- and hardware-dependent.

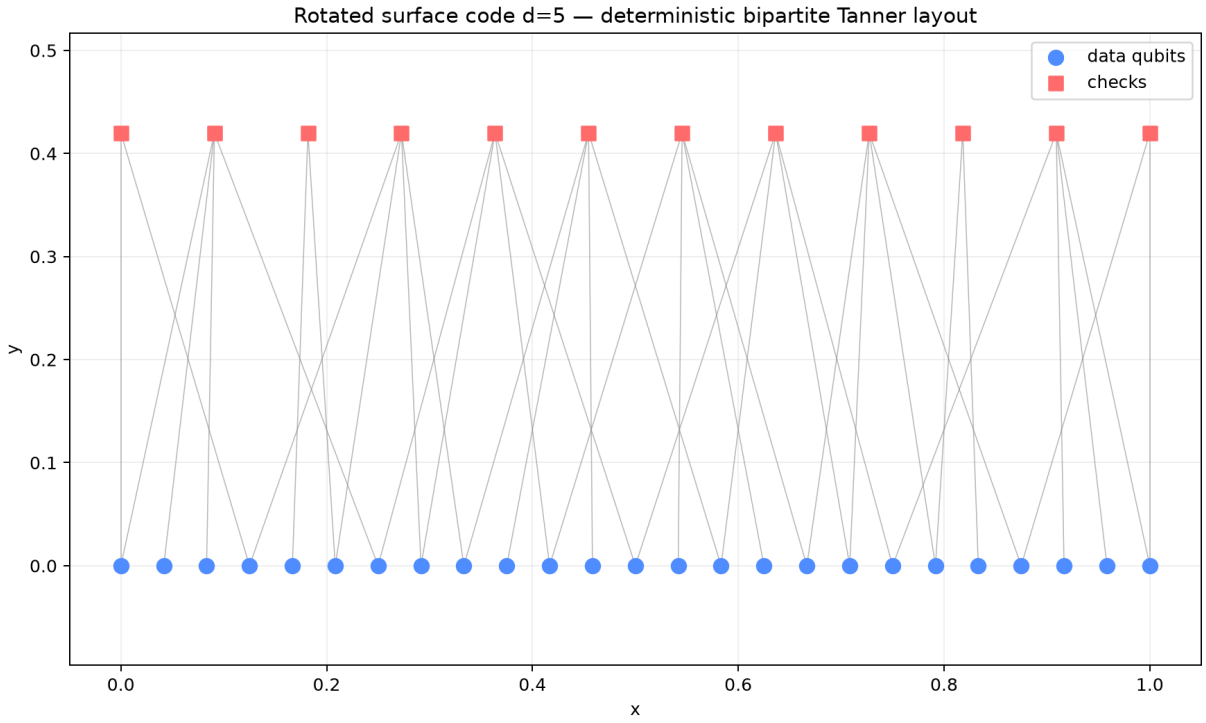
Code family properties

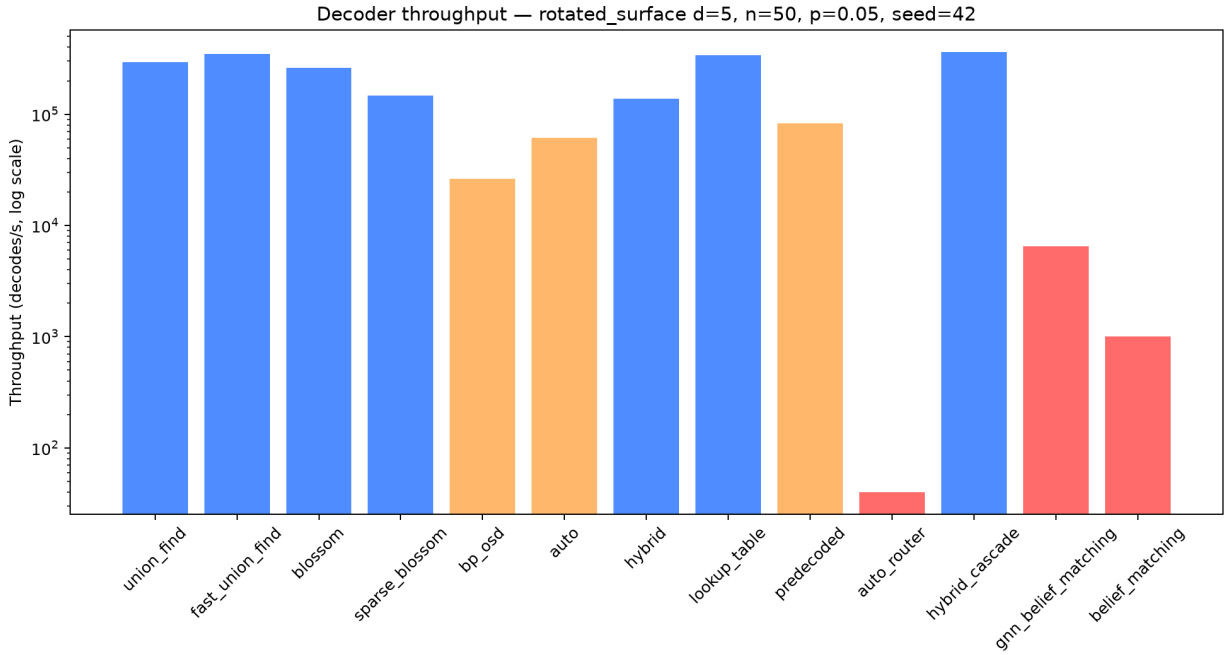
Family	Distance	n_qubits	n_checks	max_degree	compatible decoders
repetition	5	5	4	2	13
ring	5	5	5	2	13
rotated_surface	5	25	12	2	13
unrotated_surface	5	40	25	2	12
toric	5	50	25	2	12
heavy_hex	5	25	20	2	13
hypergraph_product	5	41	20	2	13
bicycle	5	10	5	2	13
bivariate_bicycle	3	72	36	3	9

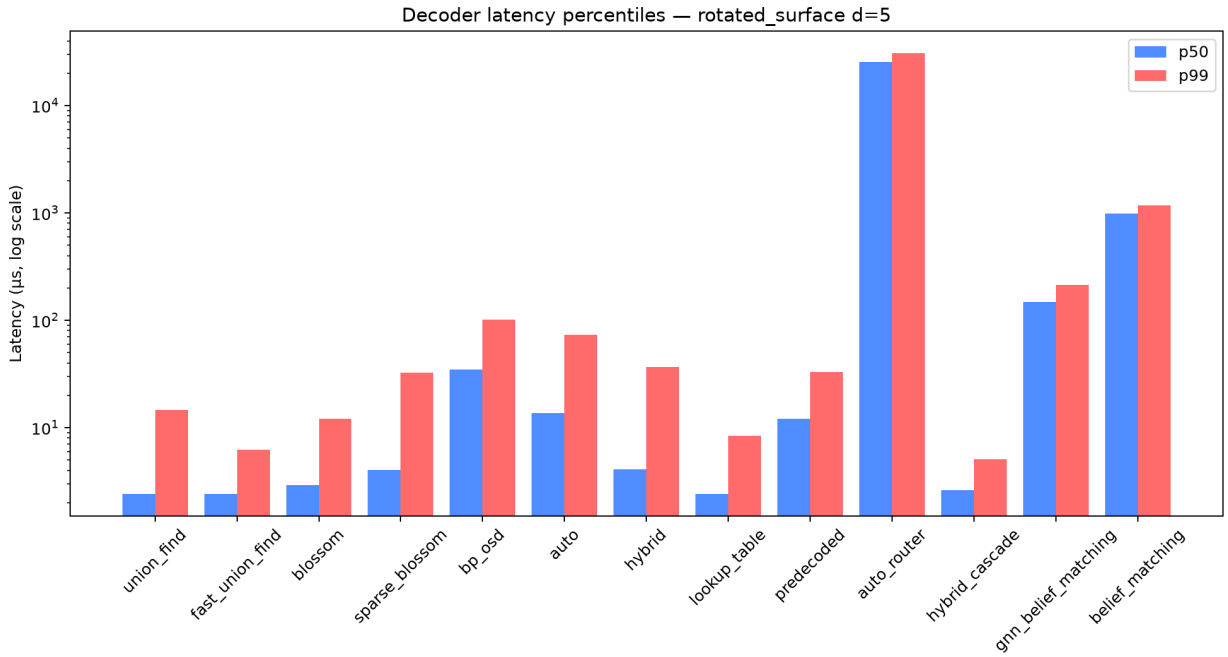
Decoder benchmark results

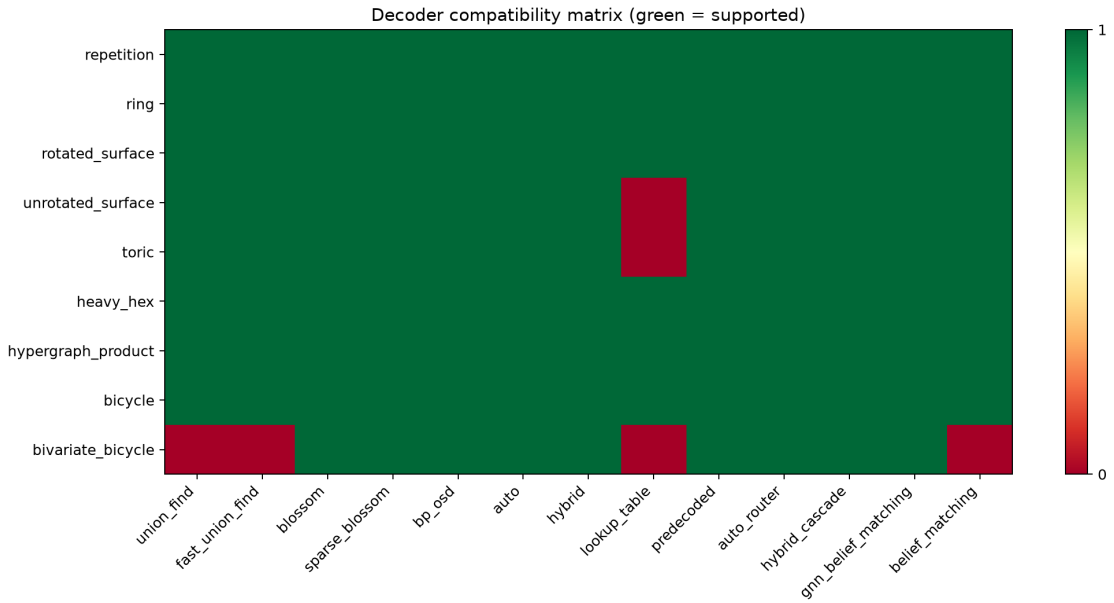
Decoder	Throughput (decodes/s)	p50 latency (µs)	p99 latency (µs)	logical error rate
union_find	295508	2.40	14.54	0.1
fast_union_find	349895	2.40	6.25	0.1
blossom	261917	2.90	12.02	0.08
sparse_blossom	146757	4.05	32.63	0.08
bp_osd	26162	34.75	101.55	0.1
auto	61125	13.60	73.22	0.1
hybrid	138812	4.10	36.70	0.08
lookup_table	337610	2.40	8.43	0.1
predecoded	82850	12.00	32.98	0.08
auto_router	40	25457.50	30758.21	0.08
hybrid_cascade	362845	2.60	5.05	0.1
gnn_belief_matching	6520	147.15	213.77	0.08
belief_matching	1001	988.05	1173.28	0.02

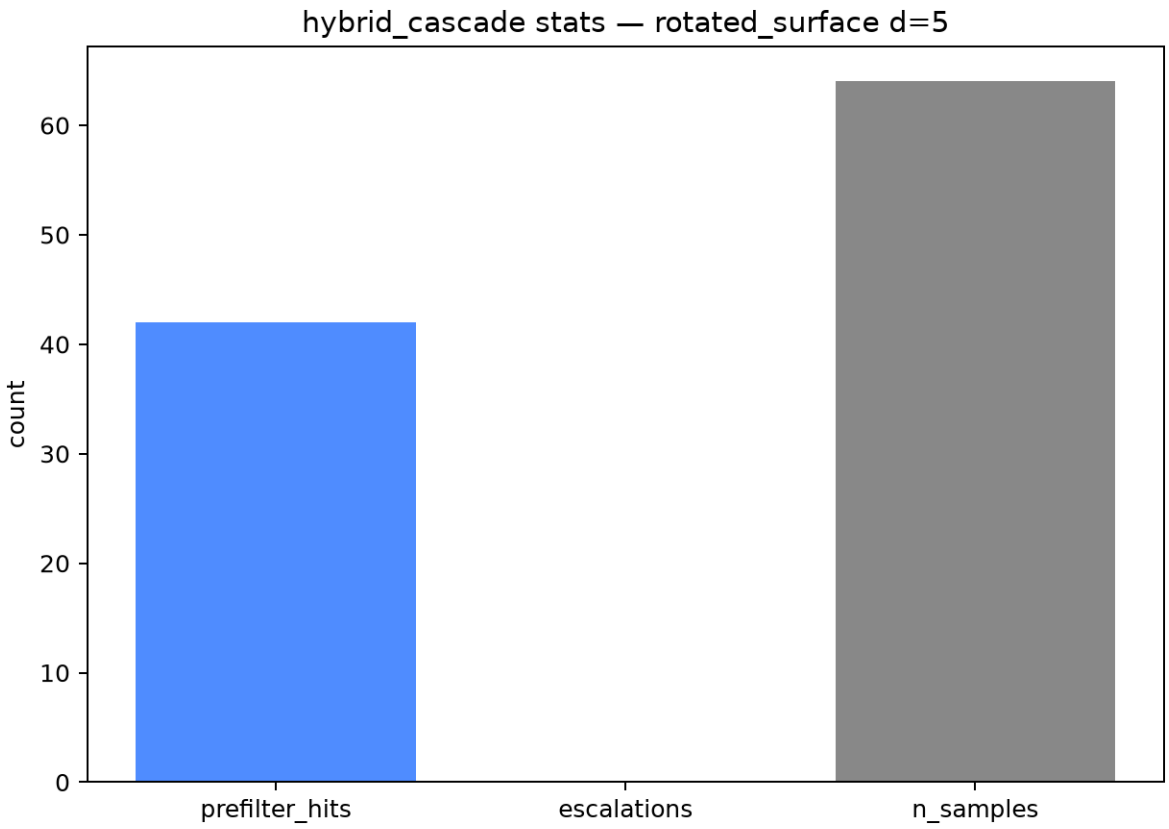
Figures


Rotated surface code d=5 — deterministic bipartite Tanner layout
data qubits
checks
y
x
0.0
0.1
0.2
0.3
0.4
0.5
0.0
0.2
0.4
0.6
0.8
1.0


Decoder throughput — rotated_surface d=5, n=50, p=0.05, seed=42
Throughput (decodes/s, log scale)
10^5
10^4
10^3
10^2
union_find
fast_union_find
blossom
sparse_blossom
bp_osd
auto
hybrid
lookup_table
predecoded
auto_router
hybrid_cascade
gnn_belief_matching
belief_matching


Decoder latency percentiles — rotated_surface d=5
p50
p99
Latency (µs, log scale)
10^4
10^3
10^2
10^1
union_find
fast_union_find
blossom
sparse_blossom
bp_osd
auto
hybrid
lookup_table
predecoded
auto_router
hybrid_cascade
gnn_belief_matching
belief_matching


Decoder compatibility matrix (green = supported)
repetition
ring
rotated_surface
unrotated_surface
toric
heavy_hex
hypergraph_product
bicycle
bivariate_bicycle
union_find
fast_union_find
blossom
sparse_blossom
bp_osd
auto
hybrid
lookup_table
predecoded
auto_router
hybrid_cascade
gnn_belief_matching
belief_matching
1
0


hybrid_cascade stats — rotated_surface d=5
count
0
10
20
30
40
50
60
prefilter_hits
escalations
n_samples

backend.py API

`backend.Any(*args, **kwargs)`

Special type indicating an unconstrained type.

`backend.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`backend.QectorError(...)`

Raised for invalid operations in the QECTOR backend.

`backend.build_code(family_key: 'str', param: 'int')`

Build a code from a family and parameter (distance).

`backend.clear_decoder_cache() -> 'bool'`

Clear the native decoder cache in qector_decoder_v3.

`backend.code_summary(code) -> 'dict[str, Any]'`

Return a summary dict for a code object.

`backend.compat_report() -> 'dict[str, Any]'`

Ecosystem-integration availability report.

`backend.compatible_decoder_kinds(code) -> 'list[str]'`

Return the decoder kinds that can actually construct on code.

`backend.compute_spring_layout(n_qubits: 'int', n_checks: 'int', check_matrix, iterations: 'int' = 100) -> 'tuple[list, list]'`

Backwards-compatible alias for :func:compute_tanner_layout.

`backend.compute_tanner_layout(n_qubits: 'int', n_checks: 'int', check_matrix) -> 'tuple[list, list]'`

Deterministic bipartite Tanner-graph layout.

`backend.deque(...)`

deque([iterable[, maxlen]]) --> deque object

`backend.flush_usage(customer_id: 'Optional[str]' = None, api_key: 'Optional[str]' = None) -> 'dict[str, Any]'`

Flush accumulated usage metrics to Stripe metered billing API.

`backend.get_code_family_info(family_key: 'str') -> 'dict[str, str]'`

Return metadata about a code family.

`backend.get_compatible_decoders(code) -> 'list[dict[str, str]]'`

Return decoder info for the decoders that can construct on this code.

`backend.get_decoder_info(kind: 'str') -> 'dict[str, str]'`

Return human-readable info about a decoder kind.

`backend.get_tanner_graph_layout(code, family: 'str', distance: 'int') -> 'tuple[list, list]'`

Return qubit and check coordinates for a clean bipartite Tanner graph.

`backend.list_available_codes() -> 'dict[str, Any]'`

Code families wired into the workbench plus the backend's native codes.list_codes() catalogue (v0.6.6). Pure introspection.

`backend.logical_failure(logicals: 'np.ndarray', error, correction) -> 'bool'`

Public: True iff residual (error+correction)%2 flips a logical.

`backend.logicals_matrix(code) -> 'Optional[np.ndarray]'`

Public accessor for the code's logical-operator matrix (or None).

`backend.make_decoder(code, decoder_kind: 'str', decoder_options: 'Optional[dict]' = None) -> 'Any'`

Public: construct a decoder of decoder_kind for code.

`backend.native_recommend(family_key: 'Optional[str]' = None, distance: 'Optional[int]' = None, n_qubits: 'Optional[int]' = None, priority: 'str' = 'balanced', batch_size: 'int' = 1) ->

'dict[str, Any]'`

Backend-native decoder recommendation (v0.6.6 recommend).

`backend.run_batch_decode(code, backend: 'str' = 'cpu', n_samples: 'int' = 100, error_rate: 'float' = 0.05, seed: 'int' = 1) -> 'dict[str, Any]'`

Run a batch decode on the given code.

`backend.run_benchmark(code, n_samples: 'int' = 1000, seed: 'int' = 42, decoder_kind: 'str' = 'union_find', error_rate: 'float' = 0.05) -> 'dict[str, Any]'`

Run a decode benchmark on the given code.

`backend.run_diagnostic_decode(code, error_rate: 'float' = 0.05, decoder_kind: 'str' = 'blossom', seed: 'int' = 42) -> 'dict[str, Any]'`

Rich single decode via v0.6.6 decode_with_diagnostics.

`backend.run_doctor_checks() -> 'dict[str, Any]'`

Run system health diagnostic checks via qd.doctor.

`backend.run_hybrid_cascade_stats(code, n_samples: 'int' = 64, error_rate: 'float' = 0.05, seed: 'int' = 1, escalation: 'Optional[str]' = None) -> 'dict[str, Any]'`

Batch-decode with HybridCascadeDecoder and expose its cascade statistics.

`backend.run_native_streaming(code, n_rounds: 'int' = 8, error_rate: 'float' = 0.03, seed: 'int' = 1, window_size: 'int' = 4) -> 'dict[str, Any]'`

Native sliding-window streaming decode (v0.6.6 sliding_window_decode).

`backend.run_neural_predecoder_training(code, n_samples: 'int' = 200, n_epochs: 'int' = 5, error_rate: 'float' = 0.05, seed: 'int' = 1) -> 'dict[str, Any]'`

Train the NeuralPredecoder on sampled (syndrome, error) pairs (lab tool).

`backend.run_parallel_batch_decode(code, n_samples: 'int' = 64, error_rate: 'float' = 0.05, seed: 'int' = 1, decoder_type: 'str' = 'union_find', n_workers: 'Optional[int]' = None) -> 'dict[str, Any]'`

Multi-process parallel batch decode via v0.6.6 DecoderPool.

`backend.run_single_decode(code, error_rate: 'float', decoder_kind: 'str', seed: 'int', decoder_options: 'Optional[dict]' = None) -> 'dict[str, Any]'`

No docstring.

`backend.run_streaming_session(code, window_size: 'int' = 5, n_rounds: 'int' = 10, error_rate: 'float' = 0.03, seed: 'int' = 1, decoder_kind: 'str' = 'union_find') -> 'dict[str, Any]'`

Run a sliding-window streaming decode session.

`backend.sample_error_and_syndrome(code, error_rate: 'float', seed: 'int')`

Public: sample one seeded error and its syndrome for code.

`backend.set_license_key_file(path: 'str') -> 'bool'`

Set license key file path for offline hard-gated verification.

`backend.sparse_blossom_radix_neighbors(code_or_checks, defects: 'list[int]', k: 'int' = 8) -> 'list[tuple[int, int, int, int]]'`

Return k-nearest candidate edges (sorted by distance) for defects via SparseBlossom RadixHeap.

`backend.validate_parameter(family_key: 'str', param: 'int') -> 'tuple[bool, str]'`

Validate a code family parameter (distance).

`backend.verify_correction(code, syndrome, correction) -> 'bool'`

Public: True iff correction reproduces the observed syndrome.

`backend.verify_license_token(token: 'str') -> 'dict[str, Any]'`

Verify an Ed25519 signed license token string.

autodebug.py API

`autodebug.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`autodebug.asdict(obj, *, dict_factory=)`

Return the fields of a dataclass instance as a new dictionary mapping field names to field values.

`autodebug.dataclass(cls=None, /, *, init=True, repr=True, eq=True, order=False, unsafe_hash=False, frozen=False, match_args=True, kw_only=False, slots=False, weakref_slot=False)`

Add dunder methods based on the fields defined in the class.

`autodebug.field(*, default=, default_factory=, init=True, repr=True, hash=None, compare=True, metadata=None, kw_only=)`

Return an object to identify dataclass fields.

`autodebug.probe_decoders(family: 'str', distance: 'int', error_rate: 'float' = 0.05, seed: 'int' = 42, verify: 'bool' = True) -> 'dict'`

Try every wired decoder against one seeded syndrome and report results.

`autodebug.resilient_batch_decode(family: 'str', distance: 'int', backend: 'str' = 'cuda', n_samples: 'int' = 100, error_rate: 'float' = 0.05, seed: 'int' = 1, fallback_chain: 'Optional[list]' = None) -> 'dict'`

Batch-decode, falling back cuda → opencl → cpu on unavailability.

`autodebug.resilient_single_decode(family: 'str', distance: 'int', error_rate: 'float' = 0.05, decoder: 'str' = 'union_find', seed: 'int' = 42, fallback_chain: 'Optional[list]' = None, verify: 'bool' = True) -> 'ResilientDecodeResult'`

Decode one seeded syndrome, falling back through decoders on failure.

`autodebug.run_self_diagnostics(probe_family: 'str' = 'repetition', probe_distance: 'int' = 3) -> 'DiagnosticsReport'`

Run a full environment/decoder/hardware self-test.

`autodebug.self_diagnostics(probe_family: 'str' = 'repetition', probe_distance: 'int' = 3) -> 'DiagnosticsReport'`

Run a full environment/decoder/hardware self-test.

hardware_routing.py API

`hardware_routing.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`hardware_routing.dataclass(cls=None, /, *, init=True, repr=True, eq=True, order=False, unsafe_hash=False, frozen=False, match_args=True, kw_only=False, slots=False, weakref_slot=False)`

Add dunder methods based on the fields defined in the class.

`hardware_routing.detect_hardware() -> 'HardwareProfile'`

Detect real GPU/CUDA/OpenCL availability via the installed package.

`hardware_routing.opencl_host() -> 'tuple[int, Optional[str]]'`

Return (device count, first platform name) exposed by the *host* OpenCL.

`hardware_routing.opencl_reason(decoder_opencl: 'bool', host_devices: 'int', host_platform: 'Optional[str]') -> 'str'`

Explain the OpenCL state in terms a user can act on.

`hardware_routing.recommend(code_family: 'Optional[str]', distance: 'Optional[int]', n_qubits: 'Optional[int]', priority: 'str') -> 'Recommendation'`

Heuristic decoder recommendation (deterministic, no model call).

version_service.py API

`version_service.Callable(*args, **kwargs)`

Deprecated alias to collections.abc.Callable.

`version_service.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`version_service.effective_app_version(prefer_latest: 'bool' = False) -> 'Optional[str]'`

The version the app presents as *its own* — the workbench baseline.

`version_service.format_version_banner(report: 'Optional[dict]' = None) -> 'str'`

One-line human banner: workbench version + installed backend version.

`version_service.get_app_version_info(refresh: 'bool' = False) -> 'dict[str, Any]'`

Local baseline for the workbench application (offline — no PyPI).

`version_service.get_backend_version_info(refresh: 'bool' = False) -> 'dict[str, Any]'`

Installed backend version, resolved locally — bundled wheel only, no PyPI.

`version_service.get_version_report(refresh: 'bool' = False) -> 'dict[str, Any]'`

Combined app + backend version report (offline — bundled wheel only).

`version_service.installed_backend_version() -> 'Optional[str]'`

The version of the compiled decoder backend actually imported, if any.

`version_service.is_newer(latest: 'Optional[str]', current: 'Optional[str]') -> 'bool'`

True iff latest is a strictly newer version than current.

`version_service.local_app_version() -> 'str'`

The workbench's baked-in baseline version (offline-safe).

`version_service.parse_version(v: 'Optional[str]') -> 'tuple'`

Parse a version string to a tuple of ints for ordering.

`version_service.resolve_versions_async(callback: 'Optional[Callable[[dict], None]]' = None, refresh: 'bool' = False) -> 'threading.Thread'`

Resolve both versions on a daemon thread, then invoke callback(report).

decoder_provisioner.py API

`decoder_provisioner.Callable(*args, **kwargs)`

Deprecated alias to collections.abc.Callable.

`decoder_provisioner.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`decoder_provisioner.abi_tag() -> 'str'`

A short tag unique to this interpreter's binary ABI.

`decoder_provisioner.activate_site() -> 'Optional[Path]'`

Place the active managed site first on sys.path (idempotently).

`decoder_provisioner.active_site() -> 'Optional[Path]'`

Read the active managed-site pointer; malformed pointers are ignored.

`decoder_provisioner.bootstrap(on_log: 'Optional[Callable[[str], None]]' = None) -> 'dict'`

Blocking pre-import gate used by main.py before backend imports.

`decoder_provisioner.ensure(prefer_latest: 'bool' = True, timeout: 'int' = 300, on_log: 'Optional[Callable[[str], None]]' = None, target_version: 'Optional[str]' = None) -> 'dict'`

Ensure an importable decoder, preferring the bundled wheel (offline).

`decoder_provisioner.ensure_async(prefer_latest: 'bool' = True, callback: 'Optional[Callable[[dict], None]]' = None, on_log: 'Optional[Callable[[str], None]]' = None, target_version: 'Optional[str]' = None) -> 'threading.Thread'`

Run :func:ensure on a daemon thread; upgrade takes effect next launch.

`decoder_provisioner.ensure_dependencies(on_log: 'Optional[Callable[[str], None]]' = None) -> 'dict'`

Check and automatically install any missing core dependencies via pip.

`decoder_provisioner.find_local_wheels() -> 'list[Path]'`

Find local or bundled .whl files for qector_decoder_v3.

`decoder_provisioner.import_ok() -> 'bool'`

True iff the decoder *actually imports* in this interpreter — i.e. its compiled extension loads. A metadata-only presence check is not enough: a wheel built for another Python ABI leaves valid dist-info but an unloadable .pyd/.so. This is the authoritative "is a usable decoder present?" test used by the boot gate.

`decoder_provisioner.is_frozen() -> 'bool'`

No docstring.

`decoder_provisioner.managed_root() -> 'Path'`

Return the app-owned, user-writable, ABI-scoped decoder storage dir.

`decoder_provisioner.purge_outdated_managed_sites(minimum_ver: 'Optional[str]' = None) -> 'list[str]'`

Delete any on-disk managed decoder site versions older than minimum_ver.

`decoder_provisioner.resolve_pip_argv() -> 'tuple[Optional[list[str]], str]'`

Find a pip interpreter that can install extension modules for this app.

`decoder_provisioner.scan_version() -> 'Optional[str]'`

Return the active managed version, otherwise a system-installed version.

`decoder_provisioner.self_check() -> 'dict'`

No docstring.

doc_generator.py API

`doc_generator.Optional(*args, **kwds)`

Optional[X] is equivalent to Union[X, None].

`doc_generator.latex_escape(value: 'Any') -> 'str'`

Escape LaTeX special characters (\ & % $ # _ { } ~ ^) in value.

MCP tool reference

56 tools via stdio JSON-RPC 2.0.

`ambiguity_cluster_decode`

Decode using AmbiguityClusterDecoder for high noise or non-graphlike codes

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- error_rate (number, default 0.05) -

- ambig_threshold (number, default 0.5) -

- max_cluster_size (integer, default 12) -

- syndrome (['array', 'null'], required) -

- seed (integer, default 42) -

`analyze_code_family`

Analyze a code family with an example code instance

Parameters

- family_name (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

`batch_decode`

Batch-decode sampled syndromes on cpu/cuda/opencl via backend.run_batch_decode

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- backend (string, default 'cpu') - One of: cpu, cuda, opencl (no silent fallback)

- n_samples (integer, default 100) -

- error_rate (number, default 0.05) -

- seed (integer, default 1) -

`batch_decode_gpu`

Batch-decode on an explicit compute backend (cpu/cuda/opencl) with honest availability reporting — unavailable GPU backends return status='unavailable' with a reason, never fake results

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 3) -

- backend (string, default 'cuda') - One of: cpu, cuda, opencl

- n_samples (integer, default 32) -

- error_rate (number, default 0.05) -

- seed (integer, default 1) -

`belief_match_decode`

Convenience seeded decode pinned to the belief_matching kind (BP posteriors + exact MWPM with faithfulness fallback)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

`benchmark_decoder`

Benchmark a decoder on a code family via backend.run_benchmark (latency percentiles, throughput, logical error rate)

Parameters

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- code_family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- error_rate (number, default 0.05) -

- n_samples (integer, default 100) -

- seed (integer, default 42) -

`check_updates`

Report whether the installed decoder backend matches the bundled release baseline (offline — no update service)

Parameters

- refresh (boolean, default False) - Accepted for compatibility; resolution is always local

`clear_decoder_cache`

Clear the backend's native decoder cache

No parameters.

`clear_results`

Clear all stored benchmark results

Parameters

- confirm (boolean, default False) -

`colour_code_decode`

Decode color code using BP-OSD over undecomposed detector error model

Parameters

- distance (integer, default 3) -

- max_iter (integer, default 30) -

- osd_order (integer, default 0) -

- syndrome (['array', 'null'], required) -

- seed (integer, default 42) -

`compare_benchmarks`

Compare stored benchmark results side by side (throughput, p99 latency, logical error rate)

Parameters

- benchmarks (array, required) - result_id values returned by the run_benchmark tool

`compat_report`

Report ecosystem-integration availability (stim/sinter/pymatching/qiskit/ldpc) and research components

No parameters.

`compatible_decoders`

Live probe: which decoder kinds construct and produce a syndrome-verified correction on this code

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 3) -

`decode_single`

Run one seeded decode and report correction weight, syndrome validity and logical failure

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

`decode_syndrome`

Decode an explicit 0/1 syndrome (length n_checks) with a chosen decoder; syndrome_valid is the GF(2) re-check, logical_failure is null (no reference error exists, so it is unknowable)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- syndrome (array, required) - 0/1 syndrome bits, length must equal the code's n_checks

- decoder_options (['object', 'null'], required) - Optional per-decoder construction options: bp_method (exact|min_sum), osd_order (0|1|2), error_rate, escalation (blossom|bposd), max_accept_weight, gnn_hidden_size, gnn_n_layers

`decode_with_options`

Seeded decode with validated per-decoder construction options (bp_osd bp_method/osd_order, hybrid_cascade escalation, GNN architecture); reports options_applied honestly

Parameters

- family (string, default 'repetition') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 3) -

- decoder_name (string, default 'bp_osd') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

- decoder_options (['object', 'null'], required) - Optional per-decoder construction options: bp_method (exact|min_sum), osd_order (0|1|2), error_rate, escalation (blossom|bposd), max_accept_weight, gnn_hidden_size, gnn_n_layers

`delete_resource`

Delete a resource by ID

Parameters

- resource_id (string, required) -

- confirm (boolean, default False) -

`diagnostic_decode`

Rich single decode via the backend's native decode_with_diagnostics (matched weight, backend used, internal fallback, timing, logicals)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- decoder_name (string, default 'blossom') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

`doctor_diagnostics`

Run system health and environment diagnostic checks via qd.doctor

No parameters.

`export_benchmark`

Export a stored benchmark result (by result_id) to the export directory

Parameters

- benchmark_id (string, required) - result_id returned by the run_benchmark tool

- format (string, default 'json') -

`flush_usage`

Flush usage metrics to Stripe metered billing API

Parameters

- customer_id (['string', 'null'], required) -

- api_key (['string', 'null'], required) -

`generate_documentation`

Generate code documentation files

Parameters

- family_key (string, default 'ring') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- param (integer, default 6) -

- formats (array, default ['json']) - Any of: json, markdown, html, latex, pdf

`get_code_properties`

Get properties of a code family

Parameters

- family_name (string, default 'ring') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

`get_config`

Get current server configuration

No parameters.

`get_decoder_info`

Get information about a decoder

Parameters

- decoder_name (string, default 'bp_osd') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

`get_hardware_info`

Get hardware/backend availability

No parameters.

`get_resource`

Get a specific resource by ID

Parameters

- resource_id (string, required) -

`get_resources`

List all resources

No parameters.

`get_results`

Get stored benchmark results (most recent first-in order)

Parameters

- limit (integer, default 10) -

`get_statistics`

Get server statistics

No parameters.

`get_system_info`

Get system information

No parameters.

`gnn_belief_match_decode`

Convenience seeded decode pinned to the gnn_belief_matching kind with optional GNN architecture overrides (gnn_hidden_size, gnn_n_layers)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

- gnn_hidden_size (['integer', 'null'], required) -

- gnn_n_layers (['integer', 'null'], required) -

`hybrid_cascade_stats`

Batch-decode through the hybrid_cascade decoder and expose its live cascade statistics (prefilter_hits, escalations, hit rate, throughput, syndrome-match rate, logical error rate)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 3) -

- n_samples (integer, default 64) -

- error_rate (number, default 0.05) -

- seed (integer, default 1) -

- escalation (['string', 'null'], required) - One of: blossom, bposd (default: backend's blossom)

`list_clients`

List registered clients

No parameters.

`list_code_families`

List available code families

No parameters.

`list_codes`

List workbench code families plus the backend's native code catalogue

No parameters.

`list_decoders`

List available decoders

No parameters.

`list_tools`

List all available MCP tools

No parameters.

`mcp_status`

Get MCP server status

No parameters.

`native_recommend`

Backend-native decoder recommendation (qector_decoder_v3.recommend) with the mapped workbench decoder_kind

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- n_qubits (['integer', 'null'], required) -

- priority (string, default 'balanced') - One of: balanced, speed, accuracy

- batch_size (integer, default 1) -

`native_streaming`

Native hardware-accelerated sliding-window streaming decode (qector_decoder_v3.sliding_window_decode) with per-round validity + telemetry

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- n_rounds (integer, default 8) -

- error_rate (number, default 0.03) -

- seed (integer, default 1) -

- window_size (integer, default 4) -

`neural_predecoder_train`

Train the NeuralPredecoder research/lab MLP on seeded (syndrome, error) pairs and evaluate on a disjoint held-out stream (exact-match, bit accuracy, syndrome validity, LER)

Parameters

- family (string, default 'repetition') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 3) -

- n_samples (integer, default 200) -

- n_epochs (integer, default 5) -

- error_rate (number, default 0.05) -

- seed (integer, default 1) -

`probe_decoders`

Probe which decoders produce a valid (syndrome-verified) correction for a code — a self-test across every wired decoder

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

`recommend_decoder`

Recommend a decoder for a code/priority using detected hardware

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- n_qubits (['integer', 'null'], required) -

- priority (string, default 'balanced') - One of: balanced, speed, accuracy

`register_client`

Register a client

Parameters

- client_id (string, required) -

- access_level (string, default 'USER') -

`reset_config`

Reset configuration to defaults

Parameters

- confirm (boolean, default False) -

`resilient_decode`

Single decode with automatic multi-decoder fallback and a full attempt trace (autodebug.resilient_single_decode)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- error_rate (number, default 0.05) -

- seed (integer, default 42) -

`run_benchmark`

Run a benchmark and store the result under a generated result_id

Parameters

- code_family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

- n_samples (integer, default 100) -

- seed (integer, default 42) -

- error_rate (number, default 0.05) -

`self_diagnostics`

Run a full environment/decoder/hardware self-diagnostics report (autodebug.run_self_diagnostics)

No parameters.

`set_config`

Merge key/value pairs into the server configuration

Parameters

- config (object, required) - Key/value pairs merged into the current configuration

`set_license_key_file`

Set license key file path for offline verification

Parameters

- path (string, required) -

`sparse_blossom_radix_neighbors`

Discover k-nearest candidate edges via SparseBlossom RadixHeap

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- defects (['array', 'null'], required) -

- k (integer, default 8) -

`stream_decode`

Run a sliding-window streaming decode session via backend.run_streaming_session

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- window_size (integer, default 5) -

- n_rounds (integer, default 10) -

- error_rate (number, default 0.03) -

- seed (integer, default 1) -

- decoder_name (string, default 'union_find') - One of: union_find, fast_union_find, blossom, sparse_blossom, bp_osd, auto, hybrid, lookup_table, predecoded, auto_router, hybrid_cascade, gnn_belief_matching, belief_matching, two_stage, ambiguity_cluster, colour_code

`two_stage_decode`

Decode using TwoStageDecoder (decoupled X/Z sector decoders)

Parameters

- family (string, default 'rotated_surface') - One of: repetition, ring, rotated_surface, unrotated_surface, toric, heavy_hex, bicycle, bivariate_bicycle, hypergraph_product, color_code

- distance (integer, default 5) -

- x_decoder (string, default 'blossom') -

- z_decoder (string, default 'blossom') -

- syndrome (['array', 'null'], required) -

- seed (integer, default 42) -

`verify_license_token`

Verify an Ed25519 signed license token string

Parameters

- token (string, required) -

`version_info`

App + decoder-backend version report (workbench baseline + installed backend, resolved locally — no network)

Parameters

- refresh (boolean, default False) - Accepted for compatibility; resolution is always local

MCP wire protocol

Newline-delimited JSON-RPC 2.0 over stdio. Launch with --mcp.

```
{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-11-05","capabilities":{},"clie
{"jsonrpc":"2.0","method":"notifications/initialized"}
{"jsonrpc":"2.0","id":2,"method":"tools/list"}
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"decode_single","arguments":{"family":"rotated_
```

Result envelope: content[0].text holds a JSON payload; isError flags tool-level failure.

Common result schemas

```
# Single decode result
{"error":[...],"syndrome":[...],"result":{"correction":[...],"hamming_weight":int,"syndrome_valid":bool,"logic
```

```
# Benchmark result
{"throughput_decodes_per_s":float,"decode_seconds":float,"n_trials":int,"p":float,"seed":int,"method":str,"bac
# Resilient decode
{"success":bool,"used_decoder":str|None,"fallback_used":bool,"syndrome_valid":bool|None,"logical_failure":bool
# Diagnostics report
{"overall_status":"pass|degraded|fail","timestamp":float,"platform":str,"python":str,"workbench_version":str,"
```

Environment variables

Variable	Effect
`QECTOR_DATA_DIR`	Relocate all QECTOR user data.
`QECTOR_SILENT`	Set to `1` to suppress the backend startup notice.
`QECTOR_LICENSE`	Ed25519 token that overrides academic/commercial for testing.
`QECTOR_DISABLE_OPENCL`	Set to `1` to skip OpenCL probing. It cannot *enable* OpenCL.
`QECTOR_ENABLE_OPENCL_AUTO`	Allows OpenCL auto-routing, but only when OpenCL is already available.

Provisioning model

The Workbench bundles the decoder wheel inside the application. On first launch it activates qector-decoder-v3 from the bundled wheel into a managed, ABI-scoped user site — fully offline, on every platform. Any outdated managed decoder left by an older release is purged automatically before activation. No internet connection, Python, or pip is required.

A splash screen is shown within roughly a second of launch and closes once the main window is mapped, so the cold start (extracting the wheel on first run, then loading a compiled extension) is never an invisible wait.

Boot diagnostics

A windowed build has no stderr, so provisioning is logged to files under the per-user data directory:

File	Contents
`logs/boot.log`	Every bootstrap step, the activated site, and the exact import error.
`logs/boot_stdio.log`	Anything written to stdout/stderr when the build has no console.

Hardware backends

Backend	Availability
`cpu`	Always available.
`cuda`	Requires an NVIDIA GPU with a healthy driver.
`opencl`	Reported unavailable by the published wheel, which is built without the OpenCL feature.

opencl_is_available() returning False is a property of the decoder build, not of the host: a machine can expose OpenCL devices and still get False, and no environment variable changes it. hardware_routing.detect_hardware() reports opencl_host_devices and

opencl_host_platform (probed from the host ICD) plus an opencl_reason string so the two situations can be told apart. Enabling the backend requires rebuilding qector-decoder-v3 with its opencl Cargo feature.

Example workflows

```
import backend as be, autodebug
code = be.build_code("rotated_surface", 5)
out = be.run_single_decode(code, error_rate=0.05, decoder_kind="blossom", seed=42)
assert out["result"]["syndrome_valid"]

out2 = be.run_single_decode(code, error_rate=0.05, decoder_kind="bp_osd", seed=7, decoder_options={"bp_method"
probe = autodebug.probe_decoders("bivariate_bicycle", 3, seed=99)
resilient = autodebug.resilient_single_decode("bivariate_bicycle", 3, decoder="union_find", seed=7)
stats = be.run_hybrid_cascade_stats(code, n_samples=64, error_rate=0.05, seed=1)
```